

Canonical Quadratic Boundary

OMI uses two quadratic forms with different responsibilities.

They must not be collapsed.

1. Frame Validation Quadratic

The first quadratic is the frame validator:

```
Q_frame(S) = E_var + E_const
```

It operates over the full eight-segment OMI frame:

```
S = [S0,S1,S2,S3,S4,S5,S6,S7]
```

It answers one question:

```
Is this 128-bit OMI envelope lawful?
```

Its return convention is:

```
Q_frame(S) = 0 → valid frame  
Q_frame(S) > 0 → invalid frame
```

This form belongs to:

```
quadratic lexer  
eBPF/XDP gate  
WASM validation  
C99 validation  
JavaScript validation  
packet/frame rejection
```

2. Coordinate Projection Quadratic

The second quadratic is the projection form:

$$Q_{xy}(x,y) = 60x^2 + 16xy + 4y^2$$

It operates over decoded local coordinates:

$x,y \in \text{local selector space}$

It answers a different question:

Where does this already-decoded state project?

Its output is used as:

$Q_{xy}(x,y) \rightarrow 0..720$ semantic sweep
 $Q_{xy}(x,y) / 6 \rightarrow 0..120$ hidden 5! root depth
 $Q_{xy}(x,y) \bmod 240 \rightarrow \text{local240 visible bridge}$

This form belongs to:

Base36 projection
emoji register projection
domino pair projection
DOM/CSSOM translate3d
local240 computation
slot5040 computation
symbolic visual encoding

3. The Boundary Rule

The frame validator must run before projection.

$Q_{\text{frame}}(S) = 0$
↓
decode local state
↓
 $Q_{xy}(x,y)$
↓
local240 / slot5040 / DOM projection

An invalid frame may not become valid merely because it has a symbolic projection.

A symbolic character may carry state, but it may not authorize state.

Therefore:

```
Q_frame(S) validates the carrier.  
Q_xy(x,y) projects the carried state.
```

4. Canonical Implementation Stub

```
// OMI Binary Quadratic Projection Form  
export const omiQuadraticProject = (x, y) =>  
  (60 * x * x) + (16 * x * y) + (4 * y * y);  
  
// Hidden root scale: 5! = 120  
export const omiRootDepth = (x, y) =>  
  omiQuadraticProject(x, y) / 6;  
  
// Visible bridge: 240-state surface  
export const omiLocal240 = (x, y) =>  
  omiQuadraticProject(x, y) % 240;  
  
// Semantic sweep: 6! = 720  
export const omiSemanticSweep = (x, y) =>  
  omiQuadraticProject(x, y);  
  
// Replay slot: 7! = 5040  
export const omiSlot5040 = (fano7, role3, x, y) =>  
  (fano7 * 720) + (role3 * 240) + omiLocal240(x, y);  
  
// Base36 symbolic projection  
export const projectBase36Symbol = (symbol) => {  
  const BASE36 = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
  const v = BASE36.indexOf(symbol.toUpperCase());  
  
  if (v === -1) return null;  
  
  const x = v & 3;  
  const y = (v >> 2) & 3;  
  const q = omiQuadraticProject(x, y);  
  
  return {  
    symbol: symbol.toUpperCase(),  
    value36: v,  
    x,  
    y,  
  }  
}
```

```
q,  
local240: q % 240,  
depth: q / 6  
};  
};
```

5. Canonical Formula Set

```
Q_xy(x,y) = 60x2 + 16xy + 4y2  
Q_xy(3,3) = 720 = 6!  
Q_xy(x,y) / 6 ∈ [0,120] = 5!  
Q_xy(x,y) mod 240 = local240  
240 = 2×5! = 15×16 = 16×16-16 = 6!/3  
5040 = 7×3×240 = 7!  
slot5040 = fano7×720 + role3×240 + local240  
Z_extrusion = Q_xy(x,y) / 6
```

6. Final Rule

Do not derive validity from projection.
Derive projection only after validity.

Or in the shortest OMI form:

Validate first.
Project second.
Replay third.
Show fourth.